

✅ SECTION – A (IN ENGLISH, C++ STYLE)

Q1(A) – Short Notes

1. Stack and its operations

Definition:

A stack is a linear data structure that follows the LIFO (Last In, First Out) principle.

Operations:

- `push(x)` – insert an element
- `pop()` – remove the top element
- `top()` – check top element
- `isEmpty()` – check if stack is empty

C++ Example

```
stack<int> s;  
s.push(10);  
s.push(20);  
cout << s.top(); // 20  
s.pop();
```

2. Queue and its Types

Definition:

A queue is a linear structure that follows the FIFO (First In, First Out) principle.

Types:

- Simple Queue
- Circular Queue
- Priority Queue
- Double Ended Queue (Deque)

C++ Example

```
queue<int> q;  
q.push(10);  
q.push(20);  
cout << q.front(); // 10  
q.pop();
```

3. Singly & Doubly Linked List

Singly Linked List:

Each node contains data + pointer to next node.

Doubly Linked List:

Each node contains data + previous pointer + next pointer.

C++ Structures

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
struct DNode {  
    int data;  
    DNode* prev;  
    DNode* next;  
};
```

Q1(B)

1. malloc() and free() – C++ Equivalent

In C++, dynamic memory is allocated using new and released using delete.

```
int *p = new int[10]; // similar to malloc
```

```
delete[] p; // similar to free
```

2. Static vs Dynamic Memory Allocation

Static Allocation	Dynamic Allocation
Size fixed at compile-time	Size decided at run-time
Stored in stack	Stored in heap
Example: <code>int a[10];</code>	Example: <code>int* a = new int[n];</code>

3. How dynamic arrays improve memory efficiency

- Memory is allocated only when needed
- No wastage
- Dynamic resizing possible
- vector in C++ resizes automatically

Example

```
vector<int> v;  
v.push_back(10);  
v.push_back(20);
```

Q2(A) – Postfix Evaluation

Expression: $5\ 3\ +\ 8\ 2\ -\ *$

Step-by-Step

1. Push 5 $\rightarrow$ [5]
2. Push 3 $\rightarrow$ [5,3]
3.
 - $\rightarrow 5+3 = 8 \rightarrow$ [8]
4. Push 8 $\rightarrow$ [8,8]
5. Push 2 $\rightarrow$ [8,8,2]
6. $- \rightarrow 8-2 = 6 \rightarrow$ [8,6]
7. $* \rightarrow 8 \times 6 = 48 \rightarrow$ [48]

✓ Final Answer: 48

C++ Code

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int evaluate(vector<string> exp) {
```

```
    stack<int> st;
```

```
    for (auto &x : exp) {
```

```
        if (isdigit(x[0])) {
```

```
            st.push(stoi(x));
```

```
        } else {
```

```
            int b = st.top(); st.pop();
```

```
            int a = st.top(); st.pop();
```

```
            if (x == "+") st.push(a + b);
```

```
            else if (x == "-") st.push(a - b);
```

```
            else if (x == "*") st.push(a * b);
```

```
            else if (x == "/") st.push(a / b);
```

```
        }
```

```
    }
```

```
    return st.top();
```

```
}
```

```
int main() {
```

```
    vector<string> exp = {"5","3","+","8","2","-","*"};
```

```
    cout << evaluate(exp);
```

```
}
```

Q2(B) – Simple Queue using Dynamic Array

Operations:

- enqueue
- dequeue
- display after each operation

C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
class DynamicQueue {
```

```
    int *arr, front, rear, size, cap;
```

```
public:
```

```
    DynamicQueue(int c = 4) {
```

```
        cap = c;
```

```
        arr = new int[cap];
```

```
        front = 0;
```

```
        rear = -1;
```

```
        size = 0;
```

```
    }
```

```
    void resize() {
```

```
        int *temp = new int[cap * 2];
```

```
        for (int i = 0; i < size; i++)
```

```
            temp[i] = arr[(front + i) % cap];
```

```
        delete[] arr;
```

```
        arr = temp;
```

```
front = 0;
rear = size - 1;
cap = cap * 2;
}
```

```
void enqueue(int x) {
    if (size == cap) resize();
    rear = (rear + 1) % cap;
    arr[rear] = x;
    size++;
    display();
}
```

```
void dequeue() {
    if (size == 0) {
        cout << "Queue empty\n";
        return;
    }
    front = (front + 1) % cap;
    size--;
    display();
}
```

```
void display() {
    cout << "Queue: ";
    for (int i = 0; i < size; i++)
        cout << arr[(front + i) % cap] << " ";
    cout << endl;
```

```

    }
};

int main() {
    DynamicQueue q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
    q.enqueue(40);
    q.enqueue(50);

    q.dequeue();
    q.dequeue();

    return 0;
}

```

Q2(B) – OR – Tower of Hanoi

Recursive Rules

1. Move $n-1$ disks from A $\rightarrow$ B
2. Move largest disk A $\rightarrow$ C
3. Move $n-1$ disks B $\rightarrow$ C

C++ Program

```

#include <iostream>

using namespace std;

void tower(int n, char A, char B, char C) {

```

```

    if (n == 1) {
        cout << "Move disk 1 from " << A << " to " << C << endl;
        return;
    }
    tower(n-1, A, C, B);
    cout << "Move disk " << n << " from " << A << " to " << C << endl;
    tower(n-1, B, A, C);
}

int main() {
    int n = 3;
    tower(n, 'A', 'B', 'C');
}

```

Q3(A) – Operations on Linked Lists

Traversal (Singly Linked List)

```

void traverse(Node* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
}

```

Traversal (Doubly Linked List)

```

void traverse(DNode* head) {
    while (head != NULL) {
        cout << head->data << " ";
        head = head->next;
    }
}

```



```
}  
}
```

Insertion at Beginning (Singly Linked List)

```
void insertBegin(Node* &head, int x) {  
    Node* temp = new Node{x, head};  
    head = temp;  
}
```

Diagram

Before: 10 → 20 → 30

After inserting 5 at beginning:

5 → 10 → 20 → 30

Q3(B) – Circular Linked List

C++ Program

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {  
    int data;  
    Node* next;  
};
```

```
Node* last = NULL;
```

```
void insertBegin(int x) {  
    Node* temp = new Node{x, NULL};  
  
    if (last == NULL) {
```

```

        last = temp;
        temp->next = temp;
    } else {
        temp->next = last->next;
        last->next = temp;
    }
}

```

```

void insertEnd(int x) {
    Node* temp = new Node{x, NULL};

    if (last == NULL) {
        last = temp;
        temp->next = temp;
    } else {
        temp->next = last->next;
        last->next = temp;
        last = temp;
    }
}

```

```

void display() {
    if (last == NULL) return;

    Node* p = last->next;

    do {
        cout << p->data << " ";
        p = p->next;
    }
}

```

```

    } while (p != last->next);

    cout << endl;
}

```

```

int main() {
    insertBegin(10);
    insertEnd(20);
    insertEnd(30);
    insertBegin(5);

    display();
}

```

Q3(B) – OR – Header Linked List

Definition:

A header linked list contains an extra node (header) which stores information or acts as a uniform starting point.

Advantages over normal singly linked list:

1. Simplifies insertion and deletion
2. Easy empty-list checking
3. No special case handling for the first node
4. Operations become uniform

Structure:

[Header] → [10] → [20] → [30]

✅ SECTION-B (FULL ENGLISH VERSION)

Q1 (A)

1. Binary Tree – Definition, Structure, Properties, Example

A binary tree is a hierarchical data structure in which every node can have at most two children, known as the left child and right child.

Structure

- Root Node – the topmost node
- Left Subtree
- Right Subtree

Properties

1. Every node can have 0, 1, or 2 children.
2. Nodes can follow a doubling pattern level-wise.
3. Depth – the length of the path from root to any leaf.
4. Height – number of edges in the longest downward path.
5. A Perfect Binary Tree has all internal nodes with 2 children.
6. A Complete Binary Tree is fully filled up to the second-last level, and the last level is filled left to right.

Example

```
    10
   / \
  20  30
 / \
40 50
```

2. Binary Search Tree (BST) – Definition, Characteristics, Example

A Binary Search Tree is a special type of binary tree where nodes are arranged according to specific ordering rules.

Key Characteristics

1. Left child value < Root value
2. Right child value > Root value
3. No duplicate elements
4. Searching, insertion and deletion operations have an average time complexity of $O(\log n)$

Example

```
    50
   /  \
  30   70
 /  \  /  \
20 40 60 80
```

Q1 (B)

Given Binary Tree

```
    25
   /  \
  15   35
 /  \  /  \
10 20 30 40
```

Inorder Traversal (Left, Root, Right)

10, 15, 20, 25, 30, 35, 40

Preorder Traversal (Root, Left, Right)

25, 15, 10, 20, 35, 30, 40

Postorder Traversal (Left, Right, Root)

10, 20, 15, 30, 40, 35, 25

Q2 (A)

Explain Linear Search, Binary Search, Interpolation Search with examples

1. Linear Search

- A simple searching method
- Each element is checked one by one
- Works on unsorted and sorted data
- Time Complexity: $O(n)$

Example

Array: 5, 8, 12, 20, 33

Searching 12 → found at index 2

2. Binary Search

- Works only on sorted arrays
- Compares the key with the middle element
- Based on divide-and-conquer
- Time Complexity: $O(\log n)$

Example

Array: 5, 8, 12, 20, 33, 40

Search 20 → middle value = 12 → search in right half → found.

3. Interpolation Search

- An improved version of binary search
- Works best on uniformly distributed data
- The mid position is estimated mathematically
- Average Time Complexity: $O(\log n)$

Example

Roll numbers: 10, 20, 30, 40, 50

Search 40 → algorithm estimates position near index 3.

Q2 (B)

Quick Sort – Manual Sorting with Steps

Given Array:

45, 12, 78, 23, 90, 34, 67, 56

Choosing 56 as pivot:

Partition

- Elements < 56: 45, 12, 23, 34
- Elements > 56: 78, 90, 67

After applying full Quick Sort recursively:

Final Sorted Array

12, 23, 34, 45, 56, 67, 78, 90

Time Complexity

- Best Case: $O(n \log n)$
- Worst Case: $O(n^2)$

Q3 (A)

Difference between BFS and DFS with example traversal

BFS (Breadth-First Search)

- Visits nodes level by level
- Uses a queue
- Finds the shortest path in an unweighted graph

Example BFS Order:

A → B → C → D → E → F

DFS (Depth-First Search)

- Travels deeper along a branch before backtracking
- Uses stack or recursion
- Does not guarantee shortest path

Example DFS Order:

A → B → D → E → C → F

Difference Table

BFS	DFS
Uses a queue	Uses stack/recursion
Level-wise traversal	Depth-wise traversal
Requires more memory	Uses less memory
Always finds shortest path	Shortest path not guaranteed

Q3 (B)

Insert the given values in an AVL Tree

Values:

20, 40, 10, 25, 35, 15

The AVL tree after all rotations and balancing:

```
    20
   /  \
  10   35
 \   /  \
 15 25 40
```

Q3 (OR)

Difference between Static Hashing and Dynamic Hashing

Static Hashing

- Fixed number of buckets
- Overflow is common
- Simple but not flexible
- Not suitable for growing data

Dynamic Hashing

- Buckets expand or shrink as data grows
- Very low overflow probability
- Uses directory-based structure
- More memory efficient and flexible

✓ SECTION–A : ANSWER–WISE NOTES (D2D STYLE)

A stack is a linear data structure that follows the LIFO (Last In First Out) principle.

In a stack, the element inserted last is removed first. All insertions and deletions take place from one end called the top.

Main Operations of Stack:

1. Push – Inserts an element at the top of the stack
2. Pop – Removes the top element from the stack
3. Peek/Top – Returns the top element without removing it
4. isEmpty – Checks whether the stack is empty

Applications of Stack:

- Expression evaluation
- Recursion
- Undo/Redo operations

Question: Define stack and explain its main operations.

A queue is a linear data structure that follows the FIFO (First In First Out) principle.

The element inserted first is removed first. Insertion is done at the rear and deletion is done from the front.

Types of Queue:

1. Simple Queue
2. Circular Queue
3. Priority Queue
4. Deque (Double Ended Queue)

Applications:

- CPU scheduling
- Printer queue
- Data buffering

Question: Define queue and explain its types.

A singly linked list consists of nodes where each node contains data and a pointer to the next node.

Traversal is possible only in one direction.

A doubly linked list contains nodes with three fields:

- Data
- Pointer to previous node
- Pointer to next node

Difference between Singly and Doubly Linked List:

Singly Linked List	Doubly Linked List
One pointer	Two pointers
Less memory	More memory
One direction traversal	Two direction traversal

Question: Explain singly linked list and doubly linked list.

Dynamic memory allocation allows memory to be allocated at run time.

In C, malloc() is used to allocate memory dynamically and free() is used to deallocate it.

Example:

```
int *p = (int*)malloc(10 * sizeof(int));
```

```
free(p);
```

Advantages:

- Efficient memory usage
- Flexible size allocation

Question: Explain malloc() and free() functions with example.

Static memory allocation allocates memory at compile time, whereas dynamic memory allocation allocates memory at run time.

Difference:

Static Memory	Dynamic Memory
Compile time	Run time
Fixed size	Flexible size
Stack memory	Heap memory

Question: Differentiate between static and dynamic memory allocation.

Dynamic arrays improve memory efficiency by allocating memory only when required.

They reduce wastage and allow resizing during program execution.

Example:

Vectors in C++ dynamically resize when elements are added.

Question: How do dynamic arrays improve memory efficiency?

The postfix expression is evaluated using a stack by scanning the expression from left to right.

If an operand is found, it is pushed onto the stack. If an operator is found, two operands are popped, the operation is performed, and the result is pushed back.

Postfix Expression: 5 3 + 8 2 - *

Steps:

- Push 5
- Push 3
- $+$ $\rightarrow 5 + 3 = 8$
- Push 8
- Push 2
- $-$ $\rightarrow 8 - 2 = 6$
- $*$ $\rightarrow 8 \times 6 = 48$

Final Answer: 48

Question: Evaluate the postfix expression using stack and show all steps.

Traversal in a linked list means visiting each node exactly once.

Insertion means adding a new node at a specified position.

Insertion at Beginning (Singly Linked List):

- Create new node
- Point new node to head
- Update head

Question: Explain traversal and insertion operations on linked list with diagram.

A circular linked list is a linked list where the last node points to the first node.

There is no NULL pointer in the list.

Advantages:

- Efficient traversal
- Suitable for round-robin scheduling

Question: Write a program to implement circular linked list or explain circular linked list.

A header linked list contains a special header node at the beginning.

The header node does not store actual data.

Advantages:

- Simplifies insertion and deletion
- No special case for empty list

Question: Explain header linked list and its advantages.

✅ SECTION-B : TREE, SEARCHING, SORTING

A binary tree is a hierarchical data structure in which each node has at most two children.

Properties:

- Maximum two children
- Height determines number of levels

- Used in expression evaluation

Question: Define binary tree and explain its properties.

A Binary Search Tree (BST) is a binary tree in which the left subtree contains values less than the root and the right subtree contains values greater than the root.

Characteristics:

- No duplicate elements
- Efficient searching

Question: Define Binary Search Tree and explain its characteristics.

Tree Traversals are methods to visit all nodes of a tree.

- Inorder: Left → Root → Right
- Preorder: Root → Left → Right
- Postorder: Left → Right → Root

Question: Explain inorder, preorder and postorder traversal.

Linear search checks each element sequentially, whereas binary search divides the array into halves.

Linear Search	Binary Search
$O(n)$	$O(\log n)$
Unsorted allowed	Sorted required

Question: Differentiate between linear search and binary search.

Quick sort is a divide and conquer sorting algorithm.

It selects a pivot element and partitions the array.

Time Complexity:

- Best Case: $O(n \log n)$
- Worst Case: $O(n^2)$

Question: Explain quick sort with example and time complexity.

Breadth First Search (BFS) uses a queue and visits nodes level by level.

Depth First Search (DFS) uses a stack or recursion and visits depth-wise.

Question: Differentiate between BFS and DFS.

IMP TOPICS FOR D2D EXAM (MONDAY)

1 STACK (High Priority – 8–10 marks)

- Definition & LIFO principle
- Push, Pop, Peek/Top, isEmpty operations
- Postfix evaluation (steps + example)
- Applications of stack (polish notation, recursion)

2 QUEUE (High Priority – 6–8 marks)

- Definition & FIFO principle
- Types: Simple, Circular, Priority, Deque (know at least one diagram)
- Operations: enqueue, dequeue, front, rear
- Circular queue vs simple queue (concept + diagram)

3 LINKED LIST (High Priority – 6–8 marks)

- Singly vs Doubly Linked List
- Traversal & insertion (at beginning, at end)
- Circular Linked List OR Header Linked List (diagram + operations)

4 TREES (Medium Priority – 6 marks)

- Binary Tree: definition + properties
- Binary Search Tree: definition + properties
- Traversals: Inorder, Preorder, Postorder
- Simple example diagrams

5 SEARCHING & SORTING (Medium Priority – 4–6 marks)

- Linear Search & Binary Search (difference + examples)
- Quick Sort OR Merge Sort (steps + final sorted array + time complexity)
- Interpolation Search (optional, bonus)

6 SHORT THEORY / EASY MARKS (Low Effort – 3–4 marks)

- Static vs Dynamic memory allocation
- malloc/free vs new/delete
- BFS vs DFS (table difference)
- Time complexity basics ($O(n)$, $O(\log n)$, $O(n^2)$)

KEY TAKEAWAY

Focus **top-down**:

1. **Stack + Queue** → guaranteed marks
2. **Linked List** → diagrams + insertion steps
3. **Tree Traversals** → small diagram = marks
4. **Searching & Sorting** → steps + one example
5. **Short Notes** → fill easy marks

This alone is enough to hit **28+ marks** without touching Red-Black Trees, Hashing internals, or complex graphs.